

Edited by Ilyana Anderson and Yue Zhao

Authored by Vinh H Nguyen

Note that the Ed links below were from last semester and thus cannot be opened. However, the broken links do not affect the main message of the post. If you have questions, search on Ed first (use the filter).

HW4 - "The Twin"

Structural twin: HW3.

Pipeline twin: the hidden one.

~~Please make sure you are using the homework_4_v4 or later for training. The valid data and HxW dimensions in generate_qa() have just been updated.~~ (Yue: the summer version addressed this)

Hi guys,

I have just finished the HW4, and would like to share some quick tips with you.

The HW4 is similar to the HW3 with base model, data generation and fine-tuning. However, the staff had the based model and fine-tuning code completed, and we just need to complete the data generation part, it's pretty much easy and straightforward with dict data and may require some basic understanding of the game/data, I think.

I don't have a basic understanding of the game/data, so I think I spent 1 hour understanding the data and the game and another 1-2 hours completing the data generation. Then, it's just mostly tuning the data pipeline and the parameters. As long as you understand the data structure, you can complete this HW easily.

A. The Lecture:

- The most related ones for me are:
 - **Lecture 5.2: Object Detection -> Simpler Object Detection, Objects as Points**
 - **Lecture 5.5: Vision Language Models.**
 - **Lecture 5.7: Current Vision Language Models -> SpatialVLM.**

B. GPU and run time benchmark:

- We may also need to modify the batch_size to better fit our GPU if necessary. [#934](#) and [here](#) for Macbook.
- The number of samples that I generated is around 460k (exclude images with no ego kart ~ 0 kart in the image)/300k (exclude all 0 answers) for your reference.
- Resume Training from saved checkpoints: [#1032](#)

The benchmark across GPU/platform that I tried for your reference (see more in the comment section).

GPU	Platform	Epochs	Approx. Time	Valid data	Accuracy	Score
RTX4060	Laptop	0.05	~ 1 hour	old	0.68	97
RTX4060	Laptop	0.20	~ 4.5 hour	new	0.79	100
T4	Colab Free Tier	0.05	~ 1 hour	new	0.69	99
A100	Colab Pro	0.50	~ 2 hours	new	(*) 0.85	105
A100	Colab Pro	0.25	~ 1 hour	new	0.79	100
L40S	Lightning.ai	0.35	~ 2 hours	old	0.80	100

(*) The accuracy fluctuates between 0.8416 - 0.85 when I have the model graded again.

FYI - I tried to submit 3 different models that have 0.85, 0.80 and 0.79 accuracy with local grader, and got around 0.77-0.78 with online grader. Got 100/105 online.

The Goldilocks

Well, when I trained on my RTX4060, I had to modify the batch_size to fit my GPU. When I trained with A100, I increased the batch_size to not to waste the compute. In both cases, I adjusted the learning rate linearly, and it can work well but not the best. My last try, I run with default hyperparameters for 1 epoch, and I got 85.83% accuracy locally and 85.42% online. For 0.05 epochs it's enough to get 99-100 points. So, try to use Colab Free Tier for 0.05 epochs, or run more epochs with default hyperparameters, even if you use A100. Lightning Free Tier A10G or L40S are good alternatives. You can run several 0.1 or 0.2 epochs without needing to pay by using UT Austin edu email account.

```
[DEBUG 00:24:597] 103
[DEBUG 00:24:597] 120
[DEBUG 00:24:597] accuracy is: 0.8583333333333333
[WARNING 00:24:604] - Test the answer accuracy [ 105 / 100 ]

[INFO 00:02:206] VLM Model Grader
[INFO 00:35:192] Testing accuracy is: 0.854167
[WARNING 00:35:206] - Test the answer accuracy [ 105 / 100 ]
```

C. The commands:

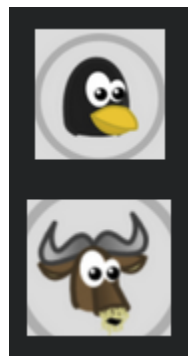
- **Demo:**
 - Train: `python -m homework.finetune demo_train`
 - Test: `python -m homework.finetune test ../demo_train`
- **QA generation check:**

- `python -m homework.generate_qa check --info_file data/valid/000c2_info.json --view_index 6`
- **QA mass generation:**
 - I write a new function `generate_all` and put it in Fire.
 - `python -m homework.generate_qa generate_all`
- **Actual Training:**
 - First, update your `output_dir` in the `train()` to `"homework/vlm_model"`
 - Train: `python -m homework.finetune train`
 - Test: `python -m homework.finetune test vlm_model`

D. The data:

- Understanding the data is the key and only requirement to do this HW in my opinion.
- In the train/valid folder, there are: `info.json` and `images` file; one info file contains information of 10 images or so.
- The important thing is to understand the `info.json` file.
- The file will have a lot of key-value pairs, for me, I just focus on the first 3: `track`, `karts` and `detections`.
 - `track`: this is the track name
 - `karts`: this is the list of kart names. (*)
 - `detections`: a list of numbers for:
 - `Object_id`: see (**)
 - `Track_id`: the name is misleading, it's actually the tracking kart id that we can use to extract the name of the kart from the `karts` list.
 - The next 4: the coordinators: `x1`, `y1`, `x2`, `y2`.

(*) The characters: <https://supertuxkart.net/Discover>



(**) This is the `object_id` and its corresponding box color in the `generate_qa.py` file: The `draw_detections` will detect the karts object with a red box for the *nominal* ego (center) kart and green boxes for other karts.

```
# Define object type mapping
OBJECT_TYPES = {
    1: "Kart",
    2: "Track Boundary",
```

```

3: "Track Element",
.....
}

# Define colors for different object types (BGR format)
COLORS = {
    1: (0, 255, 0), # Green for karts
    .....
}

```

Thanks to **Lombino** for figuring out that the *nominal* ego kart has the box color changed to red.

```

# Get color for this object type
if track_id == 0:
    color = (255, 0, 0)
else:
    color = COLORS.get(class_id, (255, 255, 255))

```

The **detected/actual** ego kart (***) is the one that we define again after removing tiny or out of bound karts.

(***) Ego kart: [#924](#)

See more in section H.

E. The missing question:

- Thanks to **Eric K**, who figured this out.
- There are 9 questions provided in the generate_qa.py, but there is 1 missing, and there is a significant number of this question in the valid data set. So, it's up to you to add an extra or not.

```

{
    "question": "Where is konqi relative to the ego car?",
    "answer": "back and right",
    "image_file": "valid/00044_00_im.jpg"
},

```

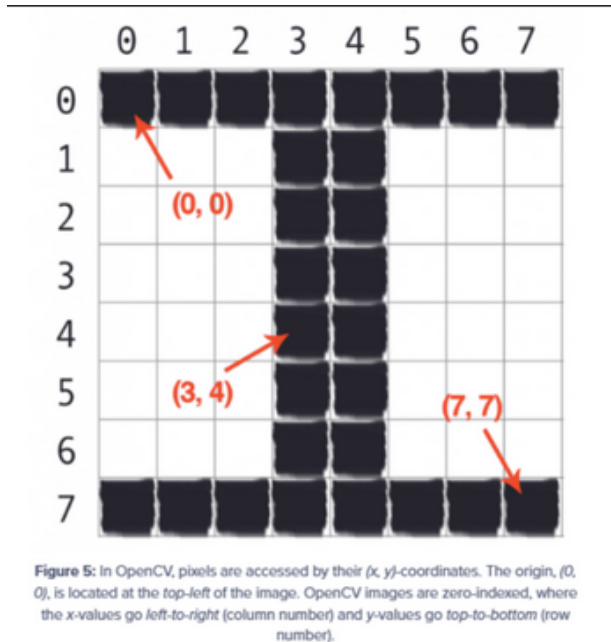
F. The working flow:

- Try to train the demo, test it and get familiar with the data and file structure.

- **Analyzing:** Read the valid data and train demo data to see how the answer looks like. The accuracy is by matching the exact text of the answer, hence, making sure you use the same answer pattern that they used (it's pretty straightforward).
- Complete the methods in generate_qa.py step by step, because the latter will depend on the former (some methods have been removed, because the staff think they are confusing and not helpful, so just have 2-3 methods left now I think). Look at the draw_detection() to see how they handle the data (scaling, rounding, etc.)
- Then, generate one qa_pairs to check and have it trained with your model to see if there is any error.
- **Auditing:** you can use your generate function to generate answers for the same questions in the valid data (120 samples in valid_grader) and make sure that they are matching. The focus of this assignment is to build the data pipeline that is aligned with the hidden pipeline used to generate the valid/test set. There is no right and wrong, just alignment. If your data pipeline is not aligned, then the model will be trained to behave differently than the expectation, and you will have low accuracy.
 - Here is my thought: [#967](#)
 - Here is the difference that we must accept and find a way to reach the way the hidden pipeline works, so that we can train the model to behave the way they want. [#969](#) (check **Eric Pham's** feedback) and [#1012](#).
 - If you just want a sanity check (not the whole 120 samples check), **see Section H.**
- Generate a bunch of qa_pairs and have it trained again. Then fine-tune your model to get good accuracy or see **the Goldilocks in section B.**

G. Hints:

- Calculate center points for: image and bboxes of the included karts (not too tiny | not out of sights). Look at the draw_detections to see how it is done. (scaling, rounding, etc.)
- Calculate Euclidean distance from each kart's center point to the image's center point -> find the closest kart-> assign it a center/ego kart.
- Compare x-values of the center points -> left|right, and y-values -> front|back based on the following coordinates. [#969](#) (check **Eric Pham's** feedback - the additional "=" at the right place can help you to push some more % alignment/accuracy) and [#1012](#).
- Calculate the % alignment of your generate_data function by comparing your generated answers for image and question in valid_grader vs the ground truth answers in valid_grader. Should get 100% or close to it before training. This step is important and helps you to save a bunch of time for training/testing the model later. *(If you just want a sanity check not the whole 120 samples check, **see Section H.**)*
- Train the model -> Fine-tune hyperparameters or see **the Goldilocks in section B.**



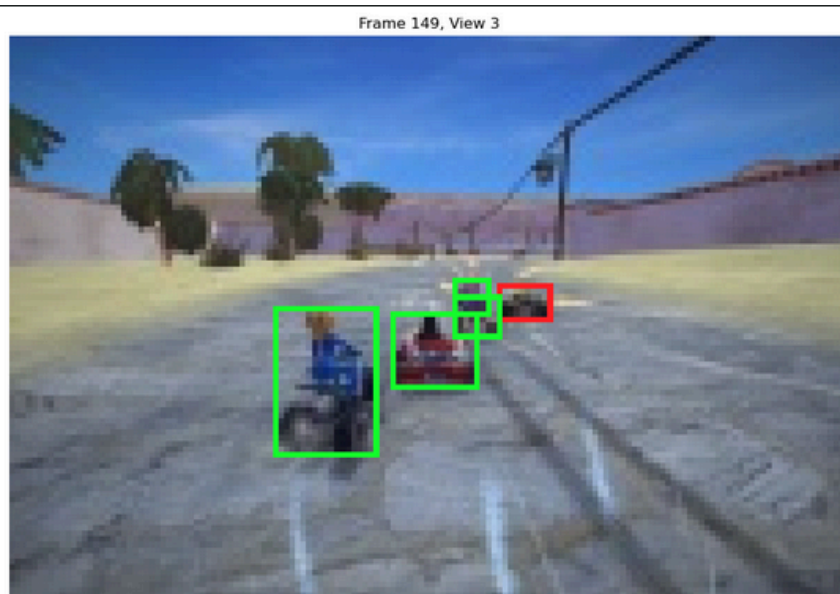
H. Example-based approach to define "back" and "front" or for a sanity check on your generate_qa_pairs()

The first 7 qa pairs in the valid_grader data are very helpful for us to learn about the answer pattern for positioning questions.

Run this: `python -m homework.generate_qa check --info_file data/valid/00095_info.json --view_index 3`

tux is the red car in the center; there are 3 car in front of tux, 1 car back (behind) tux.

sara_the_racer (the blue one) is back and left of tux (the actual/detected center/ego car).



tux:



sara:



Happy exploring!

Hope it helps!